

Women Safety App - Full Playbook

Generated: 2025-12-19

Audience: Product, Engineering, QA, Ops

1) Mission & Value

- Provide rapid emergency response, reliable communication, and proactive safety insights.
- One-tap SOS, multi-channel alerts, real-time location, AI risk checks, and a home-screen panic widget.

2) User Personas & Key Journeys

- Primary user: Needs quick SOS, share location, and trusted guardians.
- Guardians: Receive alerts, links, and updates; optionally track location.
- Journeys:
 - a) Trigger SOS → countdown → alarm → evidence capture → multi-channel alerts → call escalation.
 - b) Share my location → pick contact → send WhatsApp/SMS with Google Maps link.
 - c) Safe route → open Google Maps walking directions from current position.
 - d) Add guardians → mark primary → used by SOS and share flows.
 - e) Panic widget → tap on home screen → trigger panic intent / launch app.

3) Platform & Architecture

- Stack: Flutter (Dart) 3.9+, Firebase (Auth, Firestore, Storage, Messaging), Kotlin AppWidget, HomeWidget plugin.
- State: BLoC/Cubits (Theme, Auth, Location, SOS) + Provider DI.
- Layers:
 - Presentation: Screens, widgets, blocs.
 - Services: SOS, share, comms, AI/ML, widget, permissions, notifications, offline queue.
 - Data: Firestore, SharedPreferences/SQLite, external APIs (Agora, Maps), local assets.

4) Routing & Screens (lib/app.dart)

- Routes: / (Splash), /login, /home, /sos, /map, /profile, /settings, /revolutionary-features, /guardians, /safety-checklist.
- Key screens:
 - Profile: Quick Actions (Share My Location → /map; Revolutionary Features → /revolutionary-features; Panic Widget Setup dialog), guardians list, reset password.
 - Map: OpenStreetMap view, current location, Share FAB → contact dialog → WhatsApp/SMS.
 - Revolutionary Features: 8 advanced items; Safe Routes FAB opens Google Maps directions.
 - SOS: Activation, countdown, cancel option, escalation hooks.
 - Home: Shortcuts and status.

5) Core Features (What & How)

- SOS Orchestration (sos_service.dart): Countdown → alarm → evidence capture (photo/audio/video optional) → alerts via WhatsApp/SMS/Email/Push → call escalation to guardians.
- Location Sharing (location_share_service.dart): Geolocator to fetch GPS → Google Maps link → share via WhatsApp/SMS with fallbacks; dialogs load guardians from Firestore.
- WhatsApp Alerts (whatsapp_service.dart): Prefilled message with coords + link; per-contact send with delay and error handling.
- SMS Alerts (sms_service.dart): Compact message; tries sms:/smsto:/mms: URIs for compatibility.
- Guardians: Firestore-backed; primary flag; used by SOS and share flows.
- Revolutionary Features (8): AI danger prediction (TFLite); voice distress analysis; face recognition (ML Kit); fake call; safe routes (Maps); evidence capture; live streaming (Agora APP_ID needed); volunteer/guardian network.
- Ride/Route Safety: Ride tracking, safe journey monitoring, route deviation alerts, shake-to-SOS, voice activation.
- Panic Home Widget: AppWidgetProvider in Kotlin + HomeWidget storage; red 2x2 layout; status text + contact count; tap sends PANIC_BUTTON_CLICKED intent / launches app.
- Settings: Theme toggle, permissions, notifications; Profile reset password.

6) Key Files (high value)

- Entry: lib/main.dart (initializes Firebase, notifications, permissions, offline queue, widget; checks pending SOS and widget panic), lib/app.dart (routes, themes).
- Screens: profile_screen.dart, map_screen.dart, revolutionary_features_screen.dart, sos_screen.dart, home_screen.dart, settings_screen.dart, guardian_management_screen.dart.
- Services: sos_service.dart, location_share_service.dart, whatsapp_service.dart, sms_service.dart, panic_widget_service.dart, protection_service.dart, notification_service.dart, permissions_service.dart, ai_danger_prediction_service.dart, distress_voice_analysis_service.dart, face_recognition_service.dart, live_streaming_service.dart, offline_queue_service.dart, guardian_network_service.dart, guardian_tracking_service.dart, ride_tracking_service.dart, fake_call_service.dart.
- Models: models/app_user.dart, models/guardian.dart, models/sos_alert.dart.
- Widget (Android): android/app/src/main/kotlin/.../PanicWidgetProvider.kt; res/layout/panic_widget.xml; res/xml/panic_widget_info.xml; res/drawable/widget_background.xml; res/drawable/panic_button_bg.xml; AndroidManifest receiver registered.

7) Setup & Configuration

- Firebase: Place android/app/google-services.json (critical). For iOS, add GoogleService-Info.plist.
- Permissions: Request/accept location, camera, mic, notifications, vibration.
- Optional: Agora APP_ID in live_streaming_service.dart; TFLite model in assets/models/ for danger prediction.
- Assets: Ensure animations/models folders exist; update pubspec assets if adding models.

8) How Flows Execute (step-by-step)

- SOS: User taps SOS → countdown → alarm (alarm_service) → evidence capture (recording_service/camera) → whatsapp_service + sms_service + notification_service (push/email if configured) → call_escalation_service.
- Share Location: Map FAB → fetch guardians from Firestore → dialog → LocationShareService.shareCurrentLocation → launches WhatsApp/SMS intent with link.
- Safe Routes: Revolutionary screen FAB → gets current position → opens Google Maps directions (walking) with origin lat/lon.
- Widget Panic: HomeWidget stores data; PanicWidgetProvider receives tap intent; Flutter checks PanicWidgetService.checkPanicTrigger on startup.

9) Testing Checklist (manual)

- Profile → Quick Actions: all three entries work.
- Map → Share → contact → WhatsApp/SMS opens with Maps link.
- Revolutionary → Safe Routes → Google Maps opens.
- Widget → Add to home (launcher supports widgets) → tap triggers app/intent.
- SOS → Alerts fire (with real credentials and granted permissions).
- Guardians → CRUD works; primary flag honored.

10) Common Issues & Resolutions

- Firebase init fails: Add google-services.json to android/app and rebuild.
- WhatsApp/SMS not opening: Ensure apps installed; phone numbers include country code; check url_launcher errors.
- Location missing: Grant location permission in system settings; ensure GPS on.
- Widget not visible: Use a launcher that supports widgets; category=home_screen set in panic_widget_info.xml.

11) Build, Run, Deploy

- Debug: flutter run -d android
- Analyze: flutter analyze
- Clean & Build: flutter clean; flutter pub get; flutter build apk --release
- Install: adb install -r build/app/outputs/flutter-apk/app-release.apk

12) Security & Privacy

- No secrets in code; require user-supplied Firebase/Agora keys.
- Encourage encrypted evidence storage; avoid logging PII; use https links; validate phone numbers.

13) Future Updates & Scope

- Play Store readiness: signing, shrink/obfuscation review, privacy policy, permission rationale screens.
- Performance: Reduce APK size (prune unused ML models), lazy-load heavy features, optimize map tiles.
- Reliability: Add retry/backoff for network ops; improve offline queue with persistence; add health checks for widget updates.
- UX: Multi-language support; accessibility (voice guidance, larger tap targets); richer error toasts.
- Security: Encrypted local store for evidence queue; guardian verification links; audit trails.
- Features: In-app guardian chat; scheduled check-ins; geofenced alerts; ML model personalization for voice distress; smartwatch companion for SOS.

14) Quick References (other docs in repo)

- `project_documentation.html`: Rich visual overview.
- `CONFIGURATION_SETUP.md`, `FIREBASE_SETUP_GUIDE.md`: Setup details.
- `WIDGET_SETUP_GUIDE.md`: Widget install steps for users.
- `QUICK_TEST_GUIDE.md`: Scenario-based testing.
- `REVOLUTIONARY_FEATURES.md`: Advanced feature notes.